

Build Smarter AI Apps: Empower LLMs with LangChain

Module Cheat Sheet: Introduction to LangChain in GenAI

Package/Method	Description	Code Example
WatsonxLLM	A class from the <code>ibm_watson_machine_learning.foundation_models.extensions.langchain</code> module that creates a LangChain compatible wrapper around IBM's watsonx.ai models.	<pre>from ibm_watsonx_ai.foundation_models import from ibm_watson_machine_learning.foundation model_id = 'mistralai/mistral-8x7b-instruct parameters = { GenParams.MAX_NEW_TOKENS: 256, GenParams.TEMPERATURE: 0.2, } credentials = {"url": "https://us-south.ml. project_id = "skills-network" model = ModelInference(model_id=model_id, params=parameters, credentials=credentials, project_id=project_id) mistral_llm = WatsonxLLM(model=model) response = mistral_llm.invoke("Who is man's</pre>
Message Types	Different types of messages that chat models can use to provide context and control the conversation. The most common message types are <code>SystemMessage</code> , <code>HumanMessage</code> , and <code>AIMessage</code> .	<pre>from langchain_core.messages import HumanMe msg = mistral_llm.invoke([SystemMessage(content="You are a helpfu HumanMessage(content="I enjoy mystery n])</pre>
PromptTemplate	A class from the <code>langchain_core.prompts</code> module that helps format prompts with variables. These templates allow you to define a consistent format while leaving placeholders for variables that change with each use case.	<pre>from langchain_core.prompts import PromptTem prompt = PromptTemplate.from_template("Tell input_ = {"adjective": "funny", "topic": "c formatted_prompt = prompt.invoke(input_)</pre>
ChatPromptTemplate	A class from the <code>langchain_core.prompts</code> module that formats a list of chat messages with variables. These templates consist of a list of message templates themselves.	<pre>from langchain_core.prompts import ChatProm prompt = ChatPromptTemplate.from_messages([("system", "You are a helpful assistant ("user", "Tell me a joke about {topic}")]) input_ = {"topic": "cats"} formatted_messages = prompt.invoke(input_)</pre>

MessagesPlaceholder	A placeholder that allows you to add a list of messages to a specific spot in a ChatPromptTemplate. This capability is useful when you want the user to pass in a list of messages you would slot into a particular spot.	<pre> from langchain_core.prompts import Messages from langchain_core.messages import HumanMessage prompt = ChatPromptTemplate.from_messages([("system", "You are a helpful assistant"), MessagesPlaceholder("msgs")]) input_ = {"msgs": [HumanMessage(content="What is the capital of France?")] formatted_messages = prompt.invoke(input_) </pre>
JsonOutputParser	A parser that allows users to specify an arbitrary JSON schema and query LLMs for outputs that conform to that schema. A parser is useful for obtaining structured data from LLMs.	<pre> from langchain_core.output_parsers import JsonOutputParser from langchain_core.pydantic_v1 import BaseModel class Joke(BaseModel): setup: str = Field(description="question for the joke") punchline: str = Field(description="answer to the joke") output_parser = JsonOutputParser(pydantic_object=Joke) format_instructions = output_parser.get_format_instructions() prompt = PromptTemplate(template="Answer the user query.\n{format_instructions}\nquery: {input_variables[\"query\"]}", input_variables=["query"], partial_variables={"format_instructions": format_instructions}) chain = prompt mixtral_llm output_parser </pre>
CommaSeparatedListOutputParser	A parser used to return a list of comma-separated items. This parser converts the LLM's response into a Python list.	<pre> from langchain.output_parsers import CommaSeparatedListOutputParser output_parser = CommaSeparatedListOutputParser() format_instructions = output_parser.get_format_instructions() prompt = PromptTemplate(template="Answer the user query. {format_instructions}\nquery: {input_variables[\"subject\"]}", input_variables=["subject"], partial_variables={"format_instructions": format_instructions}) chain = prompt mixtral_llm output_parser result = chain.invoke({"subject": "ice cream flavors"}) </pre>
Document	A class from the langchain_core.documents module that contains information about some data. This class has the following two attributes: page_content (the content of the document) and metadata (arbitrary metadata associated with the document).	<pre> from langchain_core.documents import Document doc = Document(page_content="Python is an interpreted, high-level, general-purpose programming language. Python's design philosophy emphasizes code readability, which is achieved by using whitespace to format the code in a clean, structured manner.", metadata={ 'my_document_id': 234234, 'my_document_source': "About Python", 'my_document_create_time': 1680013000 }) </pre>

Chroma	A vector store from langchain.vectorstores that stores embeddings and provides methods for similarity search. You can use Chroma for storing and retrieving documents based on semantic similarity.	<pre> from langchain.vectorstores import Chroma // Create a vector store from documents docsearch = Chroma.from_documents(chunks, w // Perform a similarity search query = "Langchain" docs = docsearch.similarity_search(query) </pre>
Retrievers	Interfaces that return documents given an unstructured query. Retrievers accept a string query as input and return a list of Document objects as output. You can use vector stores as the backbone of a retriever.	<pre> # Convert a vector store to a retriever retriever = docsearch.as_retriever() // Retrieve documents docs = retriever.invoke("Langchain") </pre>
ParentDocumentRetriever	A retriever from langchain.retrievers that splits documents into small chunks for embedding but returns the parent documents during retrieval. This retriever balances accurate embeddings with context preservation.	<pre> from langchain.retrievers import ParentDocu from langchain.storage import InMemoryStore parent_splitter = CharacterTextSplitter(chu child_splitter = CharacterTextSplitter(chun vectorstore = Chroma(collection_name="split_parents", embedding_function=watsonx_embedding) store = InMemoryStore() retriever = ParentDocumentRetriever(vectorstore=vectorstore, docstore=store, child_splitter=child_splitter, parent_splitter=parent_splitter,) retriever.add_documents(documents) retrieved_docs = retriever.invoke("Langchai </pre>
RetrievalQA	A chain from langchain.chains that answers questions based on retrieved documents. The RetrievalQA chain combines a retriever with an LLM to generate answers based on the retrieved context.	<pre> from langchain.chains import RetrievalQA qa = RetrievalQA.from_chain_type(llm=mixtral_llm, chain_type="stuff", retriever=docsearch.as_retriever(), return_source_documents=False) query = "what is this paper discussing?" answer = qa.invoke(query) </pre>

ChatMessageHistory	A lightweight wrapper from langchain.memory that provides convenient methods for saving HumanMessages, AIMessages, and then fetching them all. You can use the ChatMessageHistory wrapper to maintain conversation history.	<pre> from langchain.memory import ChatMessageHis history = ChatMessageHistory() history.add_ai_message("hi!") history.add_user_message("what is the capit // Access the messages history.messages // Generate a response using the history ai_response = mixtral_llm.invoke(history.me </pre>
ConversationBufferMemory	A memory module from langchain.memory that allows for the storage of messages and conversation history. You can use this memory module conversation chains to maintain context across multiple interactions.	<pre> from langchain.memory import ConversationBu from langchain.chains import ConversationCh conversation = ConversationChain(llm=mixtral_llm, verbose=True, memory=ConversationBufferMemory()) response = conversation.invoke(input="Hello </pre>
LLMChain	A basic chain from langchain.chains that combines a prompt template with an LLM. It's the simplest form of chain in LangChain.	<pre> from langchain.chains import LLMChain template = """Your job is to come up with a {location} YOUR RESPONSE: """ prompt_template = PromptTemplate(template=t location_chain = LLMChain(llm=mixtral_llm, prompt=prompt_template, output_key='meal') result = location_chain.invoke(input={'loca </pre>
SequentialChain	A chain from langchain.chains that combines multiple chains in sequence, where the output of one chain becomes the input for the next chain. SequentialChain is useful for multi-step processing.	<pre> from langchain.chains import SequentialChai // First chain - gets a meal based on locat location_chain = LLMChain(llm=mixtral_llm, prompt=location_prompt_template, output_key='meal') // Second chain - gets a recipe based on me dish_chain = LLMChain(llm=mixtral_llm, prompt=dish_prompt_template, output_key='recipe') // Third chain - estimates cooking time recipe_chain = LLMChain(llm=mixtral_llm, prompt=recipe_prompt_template, output_key='time') </pre>

		<pre>// Combine into sequential chain overall_chain = SequentialChain(chains=[location_chain, dish_chain, rec input_variables=['location'], output_variables=['meal', 'recipe', 'ti verbose=True)</pre>
RunnablePassthrough	A component from langchain_core.runnables that allows function chaining to use the 'assign' method, enabling structured multi-step processing.	<pre>from langchain_core.runnables import Runnab // Create each individual chain with the pi location_chain_lcel = (PromptTemplate.from_template(location_t mixtral_llm StrOutputParser()) dish_chain_lcel = (PromptTemplate.from_template(dish_templ mixtral_llm StrOutputParser()) time_chain_lcel = (PromptTemplate.from_template(time_templ mixtral_llm StrOutputParser()) overall_chain_lcel = (RunnablePassthrough.assign(meal=lambda : RunnablePassthrough.assign(recipe=lambd RunnablePassthrough.assign(time=lambd) // Run the chain result = overall_chain_lcel.invoke({"locati pprint(result)</pre>
Tool	A class from langchain_core.tools that represents an interface that an agent, chain, or LLM can use to interact with the world. Tools perform specific tasks like calculations and data retrieval.	<pre>from langchain_core.tools import Tool from langchain_experimental.utilities import python_repl = PythonREPL() python_calculator = Tool(name="Python Calculator", func=python_repl.run, description="Useful for when you need to) result = python_calculator.invoke("a = 3; b</pre>
@tool decorator	A decorator from langchain.tools that simplifies the creation of custom tools. This tool automatically converts a function into a Tool object.	<pre>from langchain.tools import tool @tool def search_weather(location: str): """Search for the current weather in th # In a real application, this function i return f"The weather in {location} is c</pre>

create_react_agent	A function from langchain.agents that creates an agent following the ReAct (Reasoning + Acting) framework. This function takes an LLM, a list of tools, and a prompt template as input and returns an agent that can reason and select tools to accomplish tasks.	<pre>from langchain.agents import create_react_a agent = create_react_agent(llm=mixtral_llm, tools=tools, prompt=prompt)</pre>
AgentExecutor	A class from langchain.agents that manages the execution flow of an agent. This class handles the orchestration between the agent's reasoning and the actual tool execution.	<pre>from langchain.agents import AgentExecutor agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True, handle_parsing_errors=True) result = agent_executor.invoke({"input": "W</pre>

Author

Hailey Quach



Skills Network